



Lightweight Attention-Driven Industrial Defect Detection for Steel Surface Inspection with Adaptive Feature Enhancement

Fengyun Cao^{1,2*}, Wenwei Ye²

¹ School of Computer Science, Hefei Normal University, 230601 Hefei, China

² School of Electronic and Information Engineering, Anhui Jianzhu University, 230601 Hefei, China

* Correspondence: Fengyun Cao (caofengyun@hfnu.edu.cn)

Received: 06-01-2025

Revised: 07-25-2025

Accepted: 08-06-2025

Citation: F. Y. Cao and W. W. Ye, "Lightweight attention-driven industrial defect detection for steel surface inspection with adaptive feature enhancement," *J. Ind Intell.*, vol. 3, no. 3, pp. 146–160, 2025. <https://doi.org/10.56578/jii030302>.



© 2025 by the author(s). Licensee Acadlore Publishing Services Limited, Hong Kong. This article can be downloaded for free, and reused and quoted with a citation of the original published version, under the CC BY 4.0 license.

Abstract: Steel surface defect detection is a critical task in intelligent manufacturing, where high accuracy and real-time performance are required for reliable quality inspection. However, existing deep learning-based approaches often rely on complex architectures, leading to increased computational burden and limited applicability in industrial environments with constrained resources. To address these challenges, a lightweight detection framework is developed to improve feature representation while maintaining computational efficiency. The proposed method integrates adaptive sampling with attention-guided feature refinement to enhance multi-scale feature extraction and contextual representation. In addition, an improved regression strategy is introduced to achieve more stable localization for irregular and low-contrast defects. The network structure is further optimized through lightweight design to reduce redundant parameters and support efficient inference. Experimental results on the Northeastern University surface defect detection (NEU-DET) dataset demonstrate that the proposed approach achieves improved detection accuracy with reduced model size and computational cost compared with baseline models. The results indicate that the method provides a practical solution for real-time industrial inspection, offering a balance between accuracy and efficiency in steel surface defect detection.

Keywords: Industrial intelligence; Steel surface defect detection; Lightweight object detection; Attention mechanism; Intelligent inspection

1 Introduction

Existing deep learning-based defect detection algorithms are divided into two categories: one-stage object detection algorithms, such as the You Only Look Once (YOLO) series [1–5], and two-stage object detection algorithms, such as Region-Based-Convolutional Neural Network (R-CNN) [6], Fast R-CNN [7], and Faster R-CNN [8–11]. The advantage of one-stage object detection algorithms lies in their high computational efficiency and fast detection speed, making them suitable for real-time detection scenarios. They are commonly used in applications that require high detection speed, such as autonomous driving [12]. Two-stage algorithms require large amounts of computation and high resource demands, resulting in slower speeds compared to one-stage algorithms. However, they typically have higher detection accuracy and are suitable for applications that require high detection precision, such as medical image analysis.

In recent years, scholars have proposed a series of industrial defect detection models based on deep learning. For defects of fan blades, Liu et al. [13] proposed Deformable Feature Integration-YOLOv8 (DFI-YOLOv8), an enhanced detection model built upon the YOLOv8 framework. The method integrates deformable convolution into the backbone to improve feature representation under complex surface conditions, while an efficient multi-scale attention mechanism is employed to emphasize salient defect features. A lightweight FasterNet module is further introduced in the neck to reduce computational overhead, and a hollow Inception Simple Parameter-Free Attention Module is designed to strengthen the detection of small-scale defects. Experimental results demonstrate that the proposed approach achieves improved detection accuracy with a more efficient model structure. Wang et al. [14] built a target detection algorithm, YOLO-Residual Large Kernel, based on YOLOv5. They innovatively applied large kernel convolution to the field of defect detection, to reduce the number of convolutional overlaps while effectively

expanding the field of view. In addition, they integrated reparameterization and convolutional feedforward networks into the backbone network, thus allowing the network to accurately identify areas and categories of defects. Wang et al. [15] adopted the GhostConv module instead of regular convolution in the YOLOv8 model to reduce computational costs. They also designed a new C2fGhost module, resulting in a compressed network model with lower parameter and computational requirements. This model could be easily deployed on mobile devices, so as to achieve lightweight deployment.

While the detection models in the aforementioned industrial defect detection field pursue detection accuracy, they often overlook the significant increase in parameter and computational complexity brought about by model adjustments. Therefore, a steel surface defect detection algorithm based on an improved YOLO-Raw Wood (YOLO-RW), named Dual Enhancement Lightweight-YOLO (DL-YOLO), is proposed. YOLO-RW is an efficient steel defect detection algorithm proposed on the basis of YOLOv8n. The specific improvements are as follows:

(1) Design a Residual Depthwise Separable Convolution Block Attention Module (RDS-CBAM) hybrid attention mechanism to enable the model to identify and extract important information from images, while retaining the original image information to a certain extent to prevent the loss of key information.

(2) Incorporate the Wide Field Attention Network (WFAN) into the backbone network to reconstruct global context information and enhance the feature representation of steel surface defects.

(3) To address the increase in parameters and computational load brought about by the addition of modules, and to reduce the resource overhead of the model, we introduce depthwise separable convolution to replace the original convolution in the hybrid attention mechanism and C2f module, namely Depth-Wise Separable Convolution C2f (DS-C2f). This approach reduces parameters and computational load while barely affecting model performance.

In this paper, the algorithm further optimized detection accuracy based on YOLO-RW. This helped improve the detection accuracy of the model while controlling the overall computational load and parameter count; therefore, the overall performance of the model could be enhanced. This algorithm comprised the following core modules:

(1) The Dynamic Sampling (DySample) dynamic upsampling module was introduced to replace the conventional interpolation upsampling module used in the original model, hence dynamically adjusting the positions of sampling points, allowing them to actively align with key areas of the image, and effectively utilizing the advantage of accurately restoring small target details. The accuracy of the model could be improved along with the introduction of this module, which almost does not increase the overall burden of the model. The model was added with superior robustness and adaptive capacity to more complex and diverse detection scenarios.

(2) The Large Separable Kernel Attention (LSKA) mechanism was incorporated into the neck network. The receptive field was expanded through large-sized separable convolutional kernels, to capture long-range information and enhance the model's resistance to interference in complex backgrounds. Furthermore, this module does not incur high computational and memory usage, thus maintaining the advantage of YOLOv8's original real-time inference.

(3) The loss function has been improved by incorporating the Minimum Points Distance Intersection over Union (MPDIoU) loss function into the detection model. This loss function optimizes the corner distance based on the traditional IoU loss function, so as to address the existing gradient vanishing issue generated by the conventional IoU. This would effectively enhance the accuracy and computational efficiency of the model.

(4) The Bottleneck module was replaced with the Ghost Bottleneck module, which had lower parameter and computational requirements, to achieve model lightweighting with minimal impact on detection accuracy. This replacement could mitigate the increase in parameters and computational complexity brought about by the aforementioned module improvements.

2 Improved Modules and Their Principles

This section will first present the general model structure of the YOLO-RW steel surface defect detection algorithm, and then will primarily focus on a detailed introduction to the DL-YOLO model presented in this paper. It will sequentially introduce the improved modules within DL-YOLO, elaborating on the process and principles behind their enhancement of the model's detection performance.

2.1 Improved Model You Only Look Once-Raw Wood Based on YOLOv8

The improved YOLO-RW structure is illustrated in Figure 1. The C2f module in the backbone network and neck network was replaced by the DS-C2f module, and the WFAN module and RDS-CBAM module were added after the backbone network and neck network, respectively, to enhance the capability of the model in respect of feature extraction.

RDS-CBAM improves upon the CBAM attention mechanism [16]. Its channel attention module first performs feature compression by applying global average pooling and global max pooling to the input image, thereby obtaining two channel description vectors. Subsequently, the two vectors are inputted into a shared multi-layer perceptron to generate a channel weight vector. The channel weight vector is element-wise added and a channel attention weight matrix is generated through the Sigmoid function. This matrix is then multiplied channel-wise with the

original feature map, to highlight the features of important channels. The spatial attention module also performs two poolings on the feature map, and then concatenates the two resulting spatial feature maps to generate a dual-channel intermediate feature map. Subsequently, a 7×7 DS is used to learn spatial weights, and a spatial attention weight matrix is generated through the Sigmoid function. Finally, the feature map generated by channel attention is multiplied position-wise with this matrix, thereby emphasizing key regions. Before outputting the feature, it is connected in a residual manner with the original image to maximize the preservation of the features of the original image and avoid feature loss due to convolution.

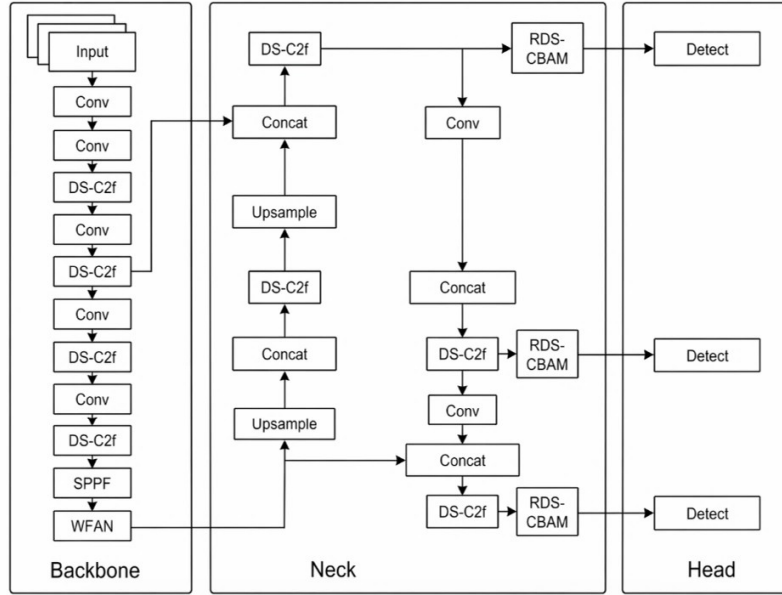


Figure 1. Improved You Only Look Once-Raw Wood (YOLO-RW) structure

Note: Conv = Convolution; DS-C2f = Depth-Wise Separable Convolution C2f; Concat = Concatenation; SPPF = Spatial Pyramid Pooling-Fast; WFAN = Wide Field Attention Network; RDS-CBAM = Residual Depthwise Separable Convolution Block Attention Module.

The WFAN enhances the model's ability to model large-scale relationships by calculating a similarity matrix, thereby improving the network's capability to identify ambiguous defects [17]. Its characteristic lies in computing weights for all positions in the image, breaking through the limitations of local receptive fields and compensating for the locality defects of traditional convolution. It directly models the relationships between feature pixels and possesses high flexibility and versatility, allowing it to be integrated into most positions within the network.

To balance model performance and lightweight requirements, the C2f module, Cross-Stage Partial with Full Concatenation, is reconstructed using DS-C2f [18]. This approach reduces parameters and computational costs while retaining multi-scale feature extraction capability. DS consists of depth-wise convolution and pointwise convolution. Depth-wise convolution performs independent convolution operations on each input channel of the image to extract spatial features from each channel. It is characterized by equal numbers of input feature map channels, convolution kernels, and output feature maps. Depth-wise convolution may result in a smaller number of feature maps, which could affect information representation. In this case, pointwise convolution is needed for the increase in dimensionality. The essence of pointwise convolution is to use a 1×1 convolution kernel to fuse channels of the image, control the number of output channels, and combine features from different channels to generate the final output feature map.

2.2 Improved Model Dual Enhancement Lightweight-You Only Look Once Based on YOLO-Raw Wood

Given that there is still room for improvement in the YOLO-RW algorithm in terms of model size and detection accuracy, an improved algorithm based on YOLO-RW, named DL-YOLO, has been designed. The upsampling module in the neck network of the YOLO-RW model has been replaced with DySample to achieve dynamic adjustment of sampling and enhance the model's multi-scale detection capability. Meanwhile, the LSKA attention mechanism has been introduced to enhance the model's capability of defect detection in complex backgrounds. The improved model structure is specifically shown in Figure 2.

2.2.1 Dynamic Sampling upsampling module

DySample is a typical data-driven lightweight dynamic upsampling module [19]. It differs from traditional upsampling methods that employ fixed-rule interpolation. Traditional upsampling techniques (such as bilinear

interpolation, nearest neighbor interpolation, deconvolution, etc.) often rely too heavily on predefined fixed weights and sampling patterns, making it difficult to achieve adaptive perception of image content when upscaling feature maps. This can easily lead to issues such as blurring of target details, loss of edge information, and excessive amplification of background noise. To address these shortcomings, DySample abandons the implementation paradigm of fixed weights in traditional upsampling, enabling adaptive adjustment of the semantic content and spatial distribution of the input feature maps through data. It dynamically calculates the sampling weights for each pixel location and its neighborhood through a point sampling mechanism, achieving differentiated aggregation and reconstruction of local image features. The process is illustrated in Figure 3.

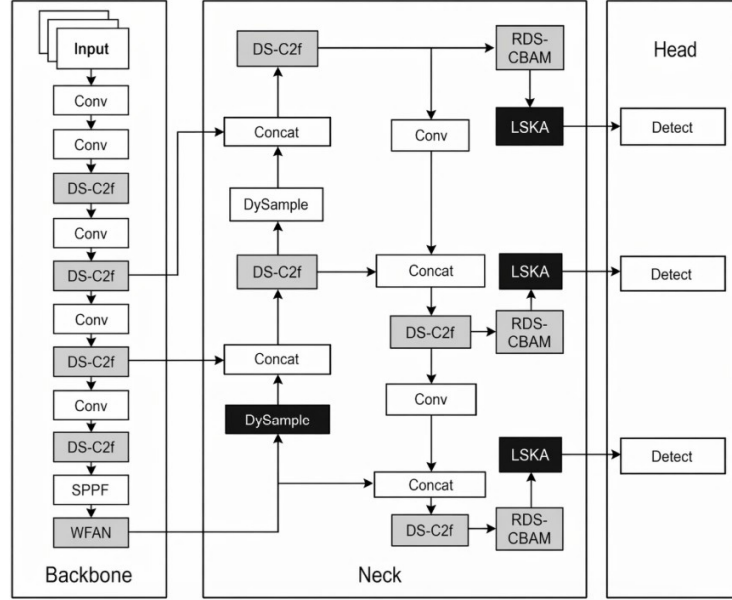


Figure 2. Dual Enhancement Lightweight-You Only Look Once (DL-YOLO) structure diagram

Note: Conv = Convolution; DS-C2f = Depth-Wise Separable Convolution C2f; Concat = Concatenation; DySample = Dynamic Sampling; SPPF = Spatial Pyramid Pooling-Fast; WFAN = Wide Field Attention Network; RDS-CBAM = Residual Depthwise Separable Convolution Block Attention Module; LSKA = Large Separable Kernel Attention.

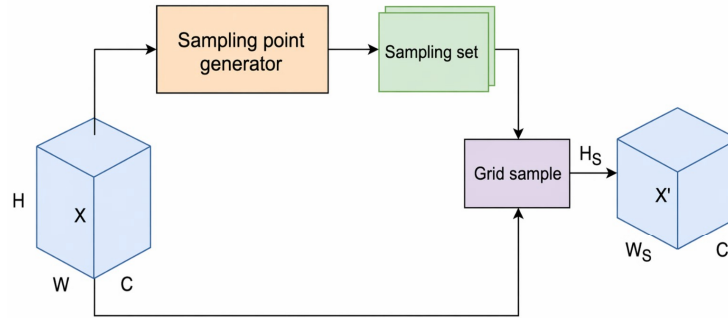


Figure 3. Flowchart of Dynamic Sampling (DySample) process

During the sampling process on the feature map, DySample can effectively preserve key details in the image. When handling tasks such as object detection and image segmentation, it significantly enhances important image structures like small targets, edge contours, and texture regions. At the same time, it can suppress redundant background and noise interference by reducing weights. In terms of overall computational cost, DySample offers lightweight advantages, enabling adaptive content perception and feature learning capabilities during the upsampling process without significantly increasing model parameter count and computational complexity. This allows the network to focus more on key target regions during scale restoration and de-emphasize non-key background regions, thereby effectively improving multi-scale feature fusion and the accuracy performance of subsequent detection tasks.

As shown in Figure 4, the core logic of DySample can be briefly divided into the following steps:

(1) Neighborhood feature extraction

Feature extraction is performed on the neighborhood positions of each pixel in the input feature map. Based on

the extraction results of its neighboring pixels, a neighborhood feature matrix is generated, ensuring that each pixel incorporates data from its surrounding pixels, providing a reliable basis for generating dynamic weights subsequently.

(2) Dynamic weight generation

This step is the core of the DySample module, where neighborhood weights are generated for each pixel position through lightweight convolution and the Softmax function, with an aim to achieve lightweight and adaptability of the module. Specifically, the following steps are involved: Firstly, the input feature map undergoes global average pooling to reduce the channel dimension; this does not only preserve the global contextual information of the image but also reduces the subsequent computational load. Subsequently, the reduced-dimension feature map is mapped to a position-wise weight matrix through 1×1 convolution. Finally, the weights are normalized using Softmax to ensure that the sum of all weights is 1, thus avoiding weight imbalance. Unlike traditional sampling methods, the exclusive weights generated at this stage reflect the importance of the candidate pixel to the current position, rather than employing the same and fixed sampling rule for every pixel position.

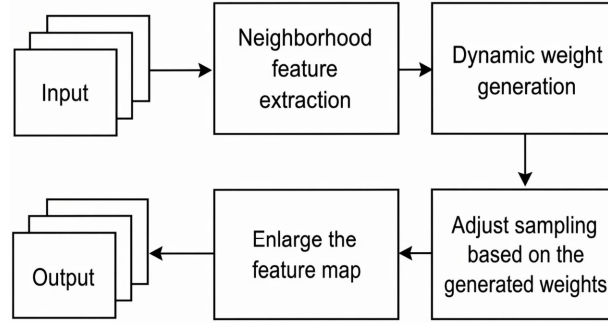


Figure 4. Schematic diagram of Dynamic Sampling (DySample) logic

(3) Weighted sampling

The sampling value for each position is obtained by multiplying and summing the neighborhood feature matrix obtained in step (1) and the dynamic weight matrix obtained in step (2). This could help achieve the goal of assigning higher weights to important pixels while reducing the weights of irrelevant pixels.

(4) Image enlargement and output

The nearest neighbor interpolation method, which balances detection accuracy and inference speed, is ultimately selected to upscale the weighted feature map. This approach enables efficient upsampling of feature details at a lower cost, while effectively reducing the difficulty of weight learning, hence making the model easier to converge and more stable during training. This design achieves an optimal trade-off between speed and accuracy, fully preserving the advantages of DySample in detail enhancement and dynamic adaptive sampling adjustment for improving accuracy, while effectively avoiding speed loss and the increased computational complexity associated with complex upsampling.

2.2.2 Large Separable Kernel Attention mechanism

To address issues such as neglecting the two-dimensional structure of images, high computational complexity, and ignoring channel adaptability in traditional self-attention mechanisms when dealing with computer vision, Guo et al. [20] proposed Large Kernel Attention (LKA) in 2022, providing a more efficient and image-processing-friendly solution for feature modeling in the field of computer vision. The core idea is as follows: Utilizing large kernel depthwise convolution and dilated depthwise convolution operations to increase the receptive field, enhancing the model's ability to perceive global image information while effectively reducing the computational and parameter overhead brought by convolution operations. Through an attention mechanism, adaptive feature selection is performed to filter out key regions of the image, capturing long-distance dependencies. The main operational process of LKA is shown in Figure 5.

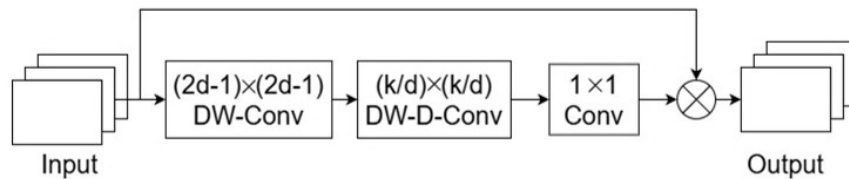


Figure 5. Schematic diagram of Large Kernel Attention (LKA) operation process

Note: DW-Conv = Depthwise Convolution; DW-D-Conv = Depthwise Dilated Convolution.

k represents the maximum receptive field at the initial setting of the module, and d denotes the dilation rate of dilated convolution. The dilation rate is used to control the spacing distance between sampling points during convolution, thereby expanding the receptive field without increasing the physical size of the convolution kernel. First, perform a $(2d-1) \times (2d-1)$ standard depth-wise convolution, followed by $\lfloor k/d \rfloor \times \lfloor k/d \rfloor$ dilated depth-wise convolution, and the size of the receptive field is selected by adjusting the dilation rate. Finally, 1×1 convolution is performed, and the convolution result is multiplied element-wise with the matrix data input into the model to obtain the final output result. Although LKA achieves the expansion of the receptive field without increasing the physical size of the convolution kernel and the computational load of the model by introducing dilated convolution, there is still room for optimization in terms of efficiency, computational load, and parameter count due to its continued use of 2D convolution.

LSKA is a lightweight attention module specifically designed for tasks related to computer vision [21]. It addresses the issue of large parameter and computational complexity arising from the direct use of $k \times k$ large kernel convolution in LKA. LSKA improves upon LKA, to continue the design of using large kernel convolution to achieve long-range feature modeling. Specifically, it replaces the large kernel convolution in the original LKA with a combination of DS and spatial decomposition convolution. In summary, its design intention stems from breaking through the bottleneck of existing visual feature extraction technology, that is, to accurately and efficiently capture long-range dependencies and multi-scale features in the input feature map while ensuring model inference efficiency and deployment feasibility.

In computer vision tasks such as image classification, object detection, and semantic segmentation, the model's understanding of global contextual information (such as spatial associations between objects in the scene and consistency across regional textures) directly depends on its ability to model long-range dependencies. The effective fusion of multi-scale features is key to enhancing the adaptability of the model to targets of different sizes, especially in detection scenarios where small and large targets coexist. However, traditional self-attention (SA) mechanisms can theoretically model global dependencies, but their computational complexity is positively correlated with the square of the feature map size ($O((H \times W)^2)$), where H and W represent the height and width of the input feature map, respectively. When dealing with high-resolution images or large-sized feature maps from deep networks, they inevitably incur high computational costs and memory usage, hence severely limiting their application effectiveness in devices with lower hardware levels or in scenarios demanding high real-time performance. Meanwhile, conventional convolution operations (such as 3×3 , 5×5 small-sized convolutional kernels) possess relatively efficient local feature extraction capabilities, and their computational complexity is only linearly correlated with the feature map size ($O(H \times W \times K^2)$), where K represents the convolution kernel size. However, due to the fixed size of the convolution kernel, its receptive field range is limited and so it is difficult to capture non-local long-range associative information in images. This results in insufficient modeling capability for global context, ultimately affecting feature extraction and representation accuracy in complex scenarios. To address these dual technical pain points, LSKA introduces an innovative "large kernel separation" design approach. Without significantly increasing computational and parameter overhead, it does not only break through the limitations of conventional small-sized convolution kernels with limited receptive fields but also avoids the computational complexity explosion issue of traditional self-attention mechanisms. This provides a solution of feature extraction for computer vision tasks that balances efficiency and performance. The schematic workflow of LSKA is shown in Figure 6.

2.2.3 Minimum Points Distance Intersection over Union loss function

The IoU loss function is a criterion used to determine detection accuracy in object detection tasks. Most detection algorithms use it as the default loss function, and its formula is shown in Eq. (1).

$$IoU = \frac{|A \cap B|}{|A \cup B|} \quad (1)$$

where, A and B represent the predicted box and the ground truth box respectively, $A \cap B$ is the intersection area between the predicted box and the ground truth box, and $A \cup B$ denotes the union area between the predicted box and the ground truth box. As can be seen from Eq. (1), the IoU loss function experiences vanishing gradients when the intersection between the ground truth box and the predicted box is empty, and it cannot distinguish the relative positional relationship between the two. When the aspect ratios of the two boxes are the same, a clear distinction cannot be made.

The MPDIoU loss function was proposed by Ma and Xu in 2023 [22]. Conventional boundary regression loss functions, such as the IoU-based loss function, often produce the same output value when facing different prediction results, which affects the convergence speed and accuracy of the final boundary box regression. Although subsequent improved versions based on the IoU loss function have improved the regression performance of prediction boxes to some extent, they still have some drawbacks: when the predicted box and the ground truth box have the same aspect ratio but differ in position and size, some loss functions struggle to perform effective regression; when detecting small targets, they lack high sensitivity to small shifts in the predicted boxes, resulting in insufficient model gradient

updates; and when the predicted box and the ground truth box do not overlap, gradient vanishing issues may occur. Therefore, based on the IoU, the MPDIoU loss function introduces a Euclidean distance parameter between the diagonal points of the boundary boxes, directly minimizing the distance between the top left and bottom right points of the predicted box and the ground truth box. It features efficient computation, stable gradients, high sensitivity to small target detection, and excellent regression performance when predicting boxes with the same aspect ratio.

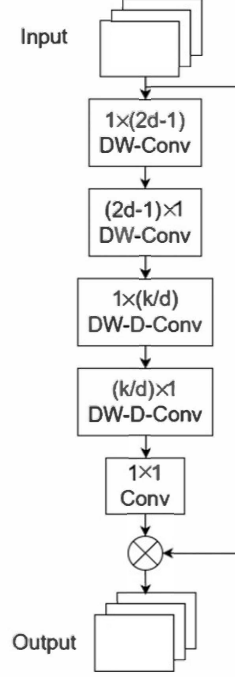


Figure 6. Schematic diagram of Large Separable Kernel Attention (LSKA) operational process

Note: DW-Conv = Depthwise Convolution; DW-D-Conv = Depthwise Dilated Convolution.

Assuming the input image size is $W \times H$, and the ground truth box B^t and the predicted box B^p are represented by the top left corner and the bottom right corner respectively, then $B^t = (x_1^t, y_1^t, x_2^t, y_2^t)$, $B^p = (x_1^p, y_1^p, x_2^p, y_2^p)$, where (x_1, y_1) is the vertex coordinate of the top left corner, and (x_2, y_2) is the vertex coordinate of the bottom right corner. To avoid the influence of image size on the distance parameters, MPDIoU uses the square of the image diagonal $W^2 + H^2$ for normalization. The measurement expression for MPDIoU at this time is: $MPDIoU = IoU - \frac{d_1^2}{W^2 + H^2} - \frac{d_2^2}{W^2 + H^2}$, and the corresponding MPDIoU loss function is defined as: $L_{MPDIoU} = 1 - MPDIoU = 1 - IoU + \frac{d_1^2 + d_2^2}{W^2 + H^2}$, where d_1 is the vertex distance between the top left corners of the ground truth box and the predicted box, and d_2 is the vertex distance between the bottom right corners of the ground truth box and the predicted box, calculated as: $d_1 = \sqrt{(x_1^p - x_1^t)^2 + (y_1^p - y_1^t)^2}$, $d_2 = \sqrt{(x_2^p - x_2^t)^2 + (y_2^p - y_2^t)^2}$. From the definition of the MPDIoU loss function, it can be seen that this loss function has the following advantages: (1) By introducing the IoU loss function value, the final output value is correlated with the degree of overlap between the predicted box and the ground truth box, to avoid the problem of the predicted box gradually deviating from the position of the ground truth box, and prioritizing the overlapped area between the two in the regression process; (2) By calculating the Euclidean distance of the diagonal vertices to penalize the offset of the predicted box, the position constraint of the diagonal vertices is achieved, thus guiding the model to finetune the predicted box and enhancing the sensitivity and positioning accuracy for small targets; and (3) By calculating the square of the image diagonal to normalize and constrain the vertex distance, the adjustment and correction of loss measurement for targets of different scales and images with different resolutions could then be solved.

2.2.4 Ghost Bottleneck structure

Compared to LKA, LSKA has a reduced computational load, but overall, it still brings a certain amount of computational and parameter requirements. In order to reduce the overall computational load and parameter count of the model, the adverse impact on the accuracy of target defect detection should be minimized. The conventional convolution in the Bottleneck model is replaced and changed to a Ghost module, thus forming the Ghost Bottleneck module.

The Ghost Bottleneck module was proposed by Noah's Ark Laboratory of Huawei [23]. Since traditional Bottleneck structure adopted standard convolution blocks to implement convolution without feature reuse, it resulted

in a high overall computational load and parameter count for the model. Ghost Bottleneck has been improved to address the aforementioned shortcomings. Based on the basic framework of Bottleneck, Ghost Bottleneck utilizes feature decoupling to generate replacements for traditional convolution, to solve the computational redundancy problem. This module specifically achieves the following upgrades:

(1) The convolution is split into two steps: the first step involves 1×1 convolution, while the second step involves linear transformation. This ensures that only $1/s$ of the feature maps in the model are generated by convolution (where s represents the transformation ratio), while the remaining feature maps are generated by linear operations, thereby reducing computational complexity.

(2) The Rectified Linear Unit (ReLU) activation function is employed during the generation of basic features, while no activation is applied during the generation of Ghost features. This approach ensures the retention of linear correlation among features to a certain extent, balancing model efficiency and accuracy.

(3) By setting the value of s , a dynamic balance between model efficiency and accuracy can be achieved, enabling it to match and complete target detection tasks under different hardware conditions.

The Ghost Module is the core component of Ghost Bottleneck, which comprises the following operational steps: Firstly, pointwise convolution is performed on the input features to generate a small number of inherent image features; secondly, linear transformation is achieved through operations such as depthwise convolution to generate ghost features; finally, the features generated in the above two steps are concatenated in the channel dimension and output. The operational steps are illustrated in Figure 7.

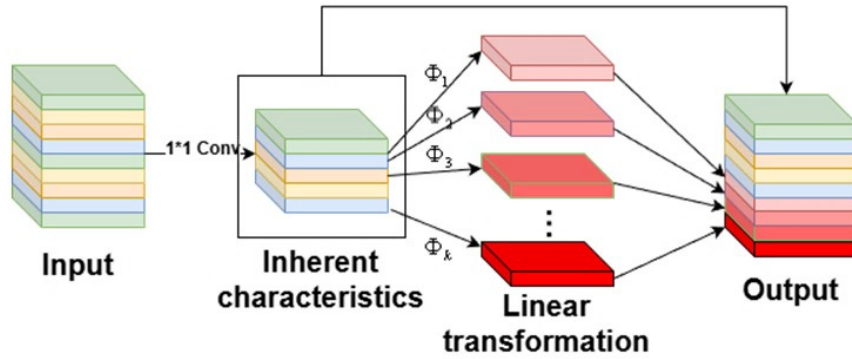


Figure 7. Schematic workflow of the ghost module

Φ_1, Φ_2, Φ_3 , and Φ_k are the inherent features of the input image. Assuming the tensor of the model input data is $c \times h \times w$, where c represents the number of channels, height, and width of the input feature map, and the tensor of the output data after one convolution is $c' \times h' \times w'$, with the conventional convolution kernel size being k and the linear transformation convolution kernel size being d , then after s transformations, the ratio of the computational complexity of the conventional convolution operation to that of the Ghost module r_s is as shown in Eq. (2).

$$\begin{aligned} r_s &= \frac{c' \times h' \times w' \times c \times k \times k}{\frac{c'}{s} \times h' \times w' \times c \times k \times k + (s-1) \times \frac{c'}{s} \times h' \times w' \times d \times d} \\ &= \frac{c \times k \times k}{\frac{1}{s} \times c \times k \times k + \frac{s-1}{s} \times d \times d} \approx \frac{s \times c}{s + c - 1} \approx s \end{aligned} \quad (2)$$

Due to the presence of $d \times d$, $k \times k$ and $s \ll c$, the ratio of the number of parameters between the regular convolution operation and the Ghost module at this time is as shown in Eq. (3).

$$r_p = \frac{c \times k \times k}{\frac{1}{s} \times c \times k \times k + \frac{s-1}{s} \times d \times d} \approx \frac{s \times c}{s + c - 1} \approx s \quad (3)$$

From Eq. (2) and Eq. (3), it can be seen that the computational and parameter counts of the Ghost module are approximately $1/s$ of those of a conventional convolutional module, thus effectively reducing the model's parameter count and computational load.

The standard Ghost Bottleneck primarily consists of a residual connection and two stacked Ghost modules, forming a feature transformation path. The first Ghost module is responsible for channel expansion and feature enhancement. It increases the number of input channels at a set expansion ratio with lower computational complexity, and generates features through convolutional blocks and linear transformations to enhance feature diversity. The introduction of a Batch Normalization (BN) layer and a ReLU activation function provides the model with certain nonlinear feature expression capabilities, thereby enhancing the model's ability to fit complex features. Additionally,

it can achieve downsampling by setting the stride size when required by the task. The second Ghost module compresses the high-dimensional features output by the previous module to match the data of the residual path in terms of the number of output channels. Subsequently, it also integrates the features through linear operations to preserve key information of the image. In this module, only BN operations are applied to the integrated features without nonlinear transformations, ensuring the integrity of residual information.

In summary, the first Ghost module is responsible for channel dimension expansion, feature enhancement, and providing nonlinearity, while the second Ghost module is responsible for channel dimension reduction, feature refinement, and maintaining linearity. This design enables the Ghost Bottleneck to effectively reduce the model's parameter count and computational complexity while maintaining target detection accuracy, thereby enhancing the model's lightweight level. The specific structure of the Ghost Bottleneck is shown in Figure 8, where Figure 8a represents the model with a stride of 1, and Figure 8b represents the model with a stride of 2.

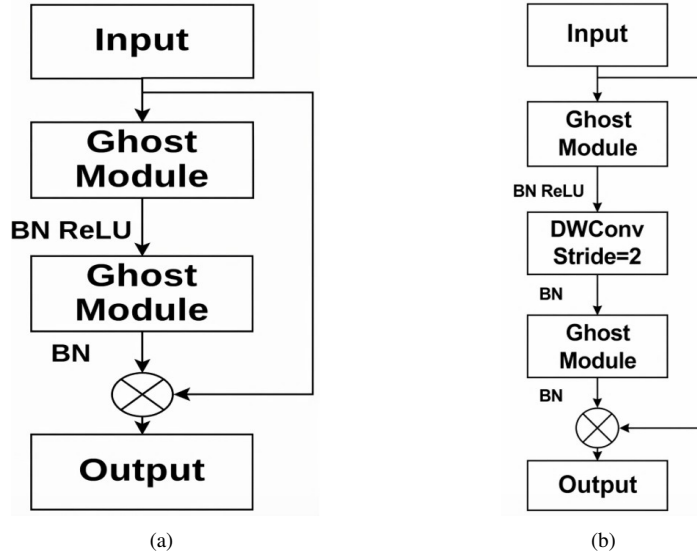


Figure 8. Schematic diagrams of two Ghost Bottleneck structures: (a) stride = 1; (b) stride = 2

3 Experimental Results and Analysis

3.1 Experimental Environment

The operating system was Windows 10; the GPU was NVIDIA GeForce GTX 1660 whereas the CPU was Intel® Core™ i7-8700 CPU @ 3.20 GHz. The experimental platform was PyCharm 2020.1.3. The development environment was Python 3.9; the Pytorch version was 1.10.1 and the CUDA framework was 11.3.

The model's parameter batch size was set to 4 and the epochs was set to 200 rounds. The initial learning rate was 0.01 and the rest were default values.

3.2 Dataset

The model used the publicly available dataset the Northeastern University surface defect detection (NEU-DET) from Northeastern University, which contained 1800 samples of steel surface defects, including these six typical types: Cracking, Inclusion, Patches, Pitted Surface, Rolled-in Scale, and Scratches. This experiment divided the dataset into training set, validation set, and testing set in a ratio of 7:2:1. Figure 9 shows examples of the six types of steel surface defects.

3.3 Analysis of Ablation Experiments

To demonstrate the effectiveness of each module, this paper used YOLO-RW as the baseline model and conducted ablation experiments on the NEU-DET dataset. The mean accuracy precision (mAP50%) at an IoU intersection to union ratio of 0.5 was used as the main evaluation metric for comparison. Floating-Point Operations Per Second (FLOPS) was a secondary evaluation metric, where “√” represented the addition of the module and “-” represented the absence of the module. Results of the specific ablation experiment are shown in Table 1.

(1) When a single DySample module was added to the YOLO-RW model for experimentation, the DySample module achieved a 0.1% accuracy improvement with almost no increase in computational complexity and parameters through dynamic upsampling.

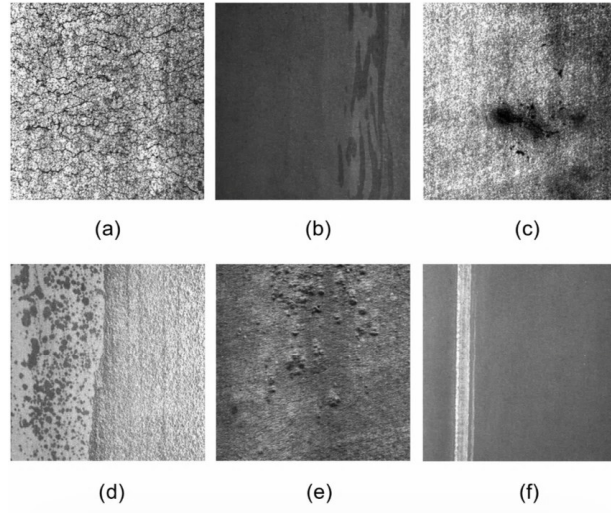


Figure 9. Sample of six defects: (a) crazing; (b) inclusion; (c) patches; (d) pitted surface; (e) rolled-in scale; (f) scratches

Table 1. Results of ablation experiments

Model	DySample	LSKA	MPDIoU	Ghost Bottleneck	Parameters/M	mAP50 (%)	FLOPs/G
YOLO-RW	-	-	-	-	3.03	76.8	7.9
+DySample	✓	-	-	-	3.04	76.9	7.9
+LSKA	-	✓	-	-	3.27	77.0	8.9
+MPDIoU	-	-	✓	-	3.03	76.9	7.9
+Ghost Bottleneck	-	-	-	✓	2.12	76.6	5.7
+DySample+LSKA	✓	✓	-	-	3.29	77.2	8.9
+DySample+LSKA+MPDIoU	✓	✓	✓	-	3.29	77.4	8.9
DL-YOLO (Proposed)	✓	✓	✓	✓	2.69	77.3	7.4

Note: YOLO = You Only Look Once; YOLO-RW = YOLO-Raw Wood; mAP50 = mean Average Precision at IoU threshold 0.5; FLOPs = Floating-Point Operations Per Second; DySample = Dynamic Sampling; LSKA = Large Separable Kernel Attention; MPDIoU = Maximum Probability Distance IoU; Ghost Bottleneck = Ghost Bottleneck (lightweight bottleneck structure); DL-YOLO = Dual Enhancement Lightweight-YOLO (the proposed model).

(2) When a single LSKA module was added to the YOLO-RW model for experimentation, the ability of the model to grasp global features of images was improved, with a 0.2% increase in accuracy. Due to the inherent size of the LSKA module, there was a slight increase in the model's computational complexity and parameters.

(3) When only replacing the original loss function with the MPDIoU loss function on the YOLO-RW model, there was no significant improvement in the model's parameters and computational complexity, but the detection accuracy improved by 0.1% compared with the baseline model. This indicates the excellent performance of the MPDIoU loss function in the face of small target defects.

(4) When only replacing the Bottleneck module with the Ghost Bottleneck module in the YOLO-RW model, the model's parameters and computational complexity decreased by 0.91 M (30.0%) and 2.2 G (27.8%), respectively, while the accuracy only decreased by 0.2% compared with the baseline model. It can be seen that the Ghost Bottleneck module plays an important role in providing lightweight models, and has almost no significant impact on the detection accuracy of the model.

(5) When the DySample upsampling algorithm and LSKA attention mechanism were simultaneously added to the YOLO-RW model, the detection accuracy of the model improved by 0.3% compared with the baseline model, despite increases in the number of model parameters and computations. When compared with the effect of the separate module, a combination of the two achieved better detection performance, indicating a positive synergistic effect between the DySample upsampling algorithm and LSKA attention mechanism.

(6) When the DySample upsampling algorithm, LSKA attention mechanism, and MPDIoU loss function were simultaneously added to the YOLO-RW model, the model accuracy was maximally improved, with a 0.6% increase compared with the baseline model. However, the parameters and computational complexity increased by 0.26 M and 1.0 G, respectively.

(7) The algorithm introduced the DySample upsampling module, LSKA attention mechanism, MPDIoU loss

function, and Ghost Bottleneck module to the YOLO-RW model. Experimental data showed that Ghost Bottleneck successfully eliminated the increase in model computation and parameter volume caused by the LSKA attention mechanism, and even the decrease was far greater than the increase in model size caused by module addition. Compared with the baseline model, it achieved a parameter reduction of 0.34 M (11.2%) and a computational reduction of 0.5 G (6.3%) on the basis of a 0.5% improvement in object detection accuracy (mAP).

Results of the ablation experiments strongly demonstrated the effectiveness of various improvement modules for the algorithm model in practical situations, and successfully achieved an improvement in the accuracy of target defect detection under the premise of controlling the computational and parameter quantities of the model.

3.4 Comparative Experimental Analysis

To further validate the actual detection capability of the model proposed in this chapter, this section compares the performance of the proposed algorithm and its related modules with other popular models of the same category on the NEU-DET dataset, aiming to demonstrate the superiority of the proposed algorithm in balancing detection accuracy and model size.

3.4.1 Experiment of overall model comparison

Firstly, the model in this paper will be compared with other popular detection models, and the experimental results are shown in Table 2. The algorithm was found to achieve 77.3% mAP50 with a parameter size of 2.69 M, and improved detection accuracy by 1.9% compared with the new generation YOLOv10 (75.4% mAP50). The parameter size of our proposed algorithm model was reduced by 0.02 M, and the computational complexity (7.4 G FLOPs) was significantly lower than that of the YOLOv10 model with 8.4 G FLOPs, hence reducing the computational complexity by 11.9% and successfully verified its balance between detection accuracy and computational complexity. Meanwhile, the algorithm DL-YOLO led to almost most popular detection models in FPS. Although it reduced 11 frames/s compared with YOLOv8n, it improved by 18 frames/s compared with the improved YOLO-RW model. Therefore, in practical applications, the algorithm in this paper could still meet the real-time requirements of industrial detection scenarios to a certain extent.

Table 2. Experimental results of overall model comparison

Model	Para./M	mAP50/%	FLOPs/G	FPS/(Frames/s)
Faster-RCNN	41.20	75.6	16.0	17
YOLOv3	12.14	62.3	19.0	43
YOLOv5	2.50	73.2	7.1	97
YOLOv6	4.24	73.5	11.9	111
YOLOv7	6.03	65.3	13.2	86
YOLOv8	3.01	74.6	8.1	148
YOLOv9	7.29	74.3	27.4	102
YOLOv10	2.71	75.4	8.4	142
YOLO-RW	3.03	76.8	7.9	119
DL-YOLO (Proposed)	2.69	77.3	7.4	137

Note: YOLO = You Only Look Once; mAP50 = mean Average Precision at IoU threshold 0.5; FLOPs = Floating-Point Operations Per Second; FPS = Frames Per Second; Para. = Parameters; DL-YOLO = Dual Enhancement Lightweight YOLO (the proposed model); Faster-RCNN = Faster Region-based-Convolutional Neural Network.

3.4.2 Loss function comparison experiment

To demonstrate the actual effectiveness of the MPDIoU loss function selected in the proposed model, a comparative experiment was conducted between the MPDIoU loss function and other popular loss functions in the YOLOv8n model, including Generalized IoU (GIoU), SCYLLA-IoU (SIoU), Efficient-IoU (EIoU), Distance IoU (DIoU), etc., under the same conditions. The results are presented in Table 3.

According to the experimental results in Table 3, it can be seen that using the MPDIoU loss function in the YOLOv8n model can effectively improve mAP50, and compared with other loss functions, it has better improvement effects on model accuracy P and recall R . This proves that the MPDIoU loss function is superior to common loss functions in terms of regression performance and has better adaptability and superiority in the task of steel surface defect detection.

3.4.3 Comparative experiment on attention mechanisms

To demonstrate the practical effectiveness of the LSKA attention mechanism in the proposed model, a comparative experiment was conducted between the LSKA attention mechanism and other commonly used attention mechanisms in the field of object detection. Squeeze-and-Excitation (SE) Module, Convolutional Block Attention Module

(CBAM), Convolutional Attention (CA), and LSKA attention mechanisms were introduced into the YOLOv8n model for performance testing. The experimental results are shown in Table 4.

According to the results in Table 4, the introduction of various attention mechanisms has a certain improvement effect on the accuracy data of YOLOv8 model object detection. However, from the ranking of the improvement results, the top two attention mechanisms are LSKA and CBAM, with 75.6% and 75.1% respectively. Although CBAM has a lower computational cost than LSKA, it is superior in terms of parameter quantity and model detection accuracy improvement. LSKA has a lower parameter quantity than CBAM by 0.07M, and mAP50% is 0.5 percentage points higher than CBAM. Although the SE and CA attention mechanisms, which rank lower in accuracy, have fewer parameters and computational complexity than LSKA, their effect on improving model detection accuracy is not significant. Therefore, from an overall perspective, LSKA attention mechanism achieves a better balance between parameters, computational complexity, and model accuracy improvement, which is in line with the design purpose of this model.

Table 3. Experimental results derived from the comparison of loss function

Loss Function	Precision (%)	Recall(%)	mAP50(%)
YOLOv8(CIoU)	73.9	65.2	74.6
GIoU	73.0	67.6	74.8
SIoU	72.9	65.4	74.2
EIoU	71.0	71.2	75.0
DIoU	74.4	63.9	74.9
MPDIoU (Proposed)	74.8	72.5	75.1

Note: YOLO = You Only Look Once; CIoU = Complete Intersection over Union; GIoU = Generalized Intersection over Union; SIoU = Scylla Intersection over Union; EIoU = Efficient Intersection over Union; DIoU = Distance Intersection over Union; MPDIoU = Maximum Probability Distance Intersection over Union (proposed loss function); mAP50 = mean Average Precision at IoU threshold 0.5.

Table 4. Comparison of the experimental results of attention mechanisms

Attention Mechanisms	Parameters/M	mAP50 (%)	FLOPs/G
YOLOv8	3.01	74.6	8.1
+SE	3.23	74.8	8.6
+CBAM	3.34	75.1	8.6
+CA	3.25	74.7	8.6
+LSKA (Proposed)	3.27	75.6	8.9

Note: YOLO = You Only Look Once; SE = Squeeze-and-Excitation; CBAM = Convolutional Block Attention Module; CA = Coordinate Attention; LSKA = Large Separable Kernel Attention (proposed attention mechanism); mAP50 = mean Average Precision at IoU threshold 0.5; FLOPs = Floating-Point Operations Per Second.

3.5 Visual Analysis

Visualization results of defect detection in the partial images of the proposed and YOLOv8n models on the NEU-DET dataset are shown in Figure 10. Among them, Figure 10a shows the partial detection results of YOLOv8n, Figure 10b displays the partial detection results of the proposed model, DL-YOLO, and Figure 10c reveals the true image labels.

It can be seen from Figure 10a that YOLOv8n had some missed detections when facing fine crack defects with large brightness changes and dense rolling oxide skin defects, which are highly similar to the background. When encountering similar defects, DL-YOLO could effectively detect the corresponding defect types. Therefore, DL-YOLO had superior detection integrity and superiority in identifying small defects with dense distribution and high similarity to the background.

Figure 11 shows typically missed detections in actual tasks of steel surface defect detection. It can be seen from the figure that the original YOLOv8n exhibited unidentifiable phenomena, as it could not meet the actual detection needs in the face of surface corrosion defects with large areas and relatively complex features. In contrast, the introduction of attention mechanism and dynamic upsampling module in DL-YOLO improved the detection capability of fuzzy defects. The proposed model could successfully predict defects and provide higher confidence and more reliable detection results. This finding further confirms that the model proposed in this paper could effectively reduce the missed detection rate of steel surface defects while improving the accuracy of practical detection. The model could thus better meet the detection requirements of small defects with unclear features and high density in industrial sites.

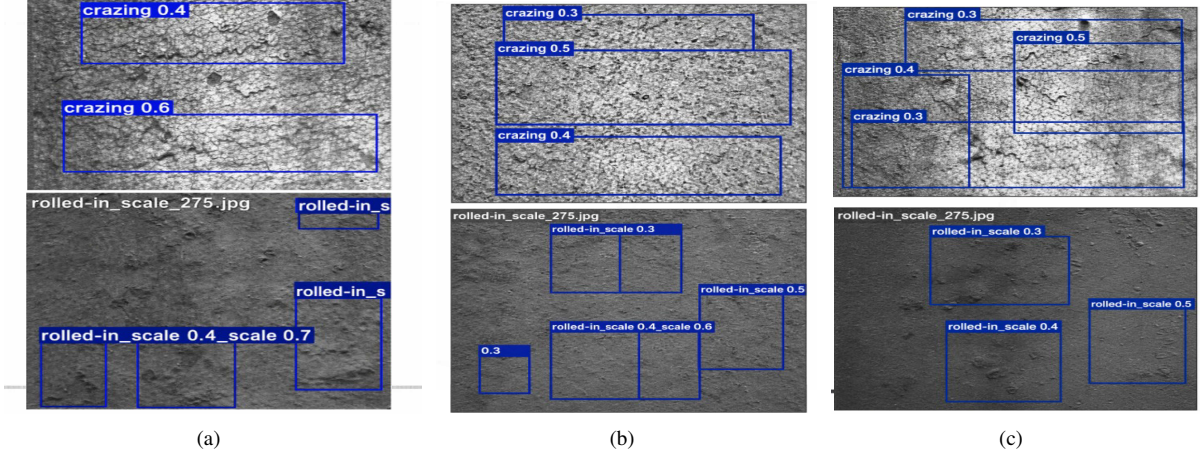


Figure 10. Comparison of visualization results: (a) You Only Look Once-v8n (YOLOv8n); (b) Dual Enhancement Lightweight-YOLO (DL-YOLO); (c) labels

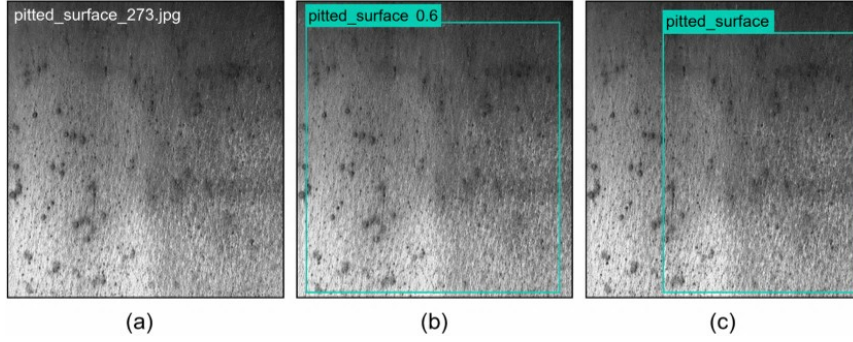


Figure 11. Photos of typically missed detection cases: (a) You Only Look Once-v8n (YOLOv8n); (b) Dual Enhancement Lightweight-YOLO (DL-YOLO); (c) labels

To sum up, in steel surface defect detection tasks that were close to practical industrial applications, the model proposed in this paper outperformed the original YOLOv8n model in terms of detection accuracy, integrity of target detection, and adaptability to complex scenarios. The proposed model demonstrated superior target defect detection performance; as a result, it could provide effective technical support for actual steel surface detection projects and has strong practical value and significance in assessing the quality of steel surface.

4 Conclusion

This article further improved YOLO-RW for the design of a steel surface defect detection algorithm, DL-YOLO. Firstly, the innovation replaced the original upsampling module with the DySample upsampling module and dynamically adjusted the sampling based on the weights of different pixels to improve sampling efficiency. Then, LSKA attention mechanism was introduced to achieve accurate and efficient capture of long-range dependencies and multi-scale features in the input feature map. By replacing the CIoU loss function of the original model with the MPDIoU loss function, the sensitivity and localization accuracy of the model for small targets were enhanced. Finally, the Bottleneck module in the original model was replaced with a more lightweight Ghost Bottleneck module to minimize the impact on model detection accuracy while improving the overall lightweight level of the model. Comparative experiments demonstrated that DL-YOLO achieved an unparalleled balance in the three indicators, i.e., parameter quantity, computational complexity, and detection accuracy. Furthermore, through comparative experiments with a single loss function and attention mechanism, it was revealed that the modules selected in the process of improving the model in this paper were superior to other choices for common module improvement. This indicates that DL-YOLO contributes to the significance and practical value in the detection of steel surface defects. Subsequent research will continue to improve the model, further enhance the performance of detection, and expand the scenarios for the applicability of the algorithm model.

Author Contributions

Conceptualization, F.Y.C. and W.W.Y.; methodology, F.Y.C.; software, F.Y.C.; validation, F.Y.C. and W.W.Y.; formal analysis, F.Y.C.; investigation, F.Y.C.; resources, F.Y.C.; data curation, W.W.Y.; writing—original draft preparation, W.W.Y.; writing—review and editing, F.Y.C.; visualization, W.W.Y.; supervision, F.Y.C.; project administration, F.Y.C.; funding acquisition, F.Y.C. All authors have read and agreed to the published version of the manuscript.

Data Availability

The data used to support the research findings are available from the corresponding author upon request.

Conflicts of Interest

The authors declare no conflict of interest.

References

- [1] A. Vijayakumar and S. Vairavasundaram, “YOLO-based object detection models: A review and its applications,” *Multimed. Tools Appl.*, vol. 83, no. 35, pp. 83 535–83 574, 2024. <https://doi.org/10.1007/s11042-024-18872-y>
- [2] Z. J. Yao, L. Zhang, and A. Khadka, “YOLOv8n-AM: Enhanced real-time smoke detection via attention-based feature interaction and multi-scale downsampling,” *J. Ind. Intell.*, vol. 2, no. 4, pp. 240–250, 2024. <https://doi.org/10.56578/jii020405>
- [3] J. Tie, C. Zhu, L. Zheng, H. Wang, C. Ruan, M. Wu, K. Xu, and J. Liu, “LSKA-YOLOv8: A lightweight steel surface defect detection algorithm based on YOLOv8 improvement,” *Alex. Eng. J.*, vol. 109, pp. 201–212, 2024. <https://doi.org/10.1016/j.aej.2024.08.087>
- [4] C. Zhao, X. Shu, X. Yan, X. Zuo, and F. Zhu, “RDD-YOLO: A modified YOLO for detection of steel surface defects,” *Measurement*, vol. 214, p. 112776, 2023. <https://doi.org/10.1016/j.measurement.2023.112776>
- [5] Y. Chen and Y. Wu, “Detection of welding defects tracked by YOLOv4 algorithm,” *Appl. Sci.*, vol. 15, no. 4, p. 2026, 2025. <https://doi.org/10.3390/app15042026>
- [6] B. Hou, “Theoretical analysis of the network structure of two mainstream object detection methods: YOLO and Fast RCNN,” *Appl. Comput. Eng.*, vol. 17, pp. 213–225, 2023. <https://doi.org/10.26914/c.cnkihy.2023.108861>
- [7] H. Yang, “Research on anti-collision of pedestrians and vehicles in hazy weather based on Fast R-CNN network,” *Farm Mach. Using Maint.*, no. 10, pp. 32–34, 2023. <https://doi.org/10.14031/j.cnki.njwx.2023.10.008>
- [8] L. Huang and J. C. Wu, “Improved Faster R-CNN method for detecting wildfire images in transmission lines,” *Mod. Electron. Tech.*, vol. 48, no. 9, pp. 173–179, 2025. <https://doi.org/10.16652/j.issn.1004-373x.2025.09.026>
- [9] U. Mittal, P. Chawla, and R. Tiwari, “EnsembleNet: A hybrid approach for vehicle detection and estimation of traffic density based on Faster R-CNN and YOLO models,” *Neural Comput. Appl.*, vol. 35, no. 6, pp. 4755–4774, 2023. <https://doi.org/10.1007/s00521-022-07940-9>
- [10] H. T. Nguyen, M. N. Nguyen, L. D. Phung, and L. T. T. Pham, “Anomalies detection in chest X-rays images using Faster R-CNN and YOLO,” *Vietnam J. Comput. Sci.*, vol. 10, no. 4, pp. 499–515, 2023. <https://doi.org/10.1142/S2196888823500094>
- [11] A. A. Lima, M. M. Kabir, S. C. Das, M. N. Hasan, and M. F. Mridha, “Road sign detection using variants of YOLO and R-CNN: An analysis from the perspective of Bangladesh,” in *Lecture Notes on Data Engineering and Communications Technologies*, Springer, Singapore, 2022, vol. 95, pp. 555–565. https://doi.org/10.1007/978-981-16-6636-0_42
- [12] P. N. Huu, Q. P. Thi, and P. T. T. Quynh, “Proposing lane and obstacle detection algorithm using YOLO to control self-driving cars on advanced networks,” *Adv. Multimedia*, vol. 2022, pp. 1–18, 2022. <https://doi.org/10.1155/2022/3425295>
- [13] B. Liu, N. Zhou, and Z. Wang, “DFI-YOLOv8 based defect detection method for fan blades,” in *Proceedings of the 2024 International Conference on Image Processing, Multimedia Technology and Machine Learning (IPMML 2024)*, Dali, China, 2025, pp. 197–202.
- [14] Y. Wang, J. Huang, S. K. Dipu, H. Zhao, S. Gao, H. Zhang, and P. Lv, “YOLO-RLC: An advanced target-detection algorithm for surface defects of printed circuit boards based on YOLOv5,” *Comput. Mater. Contin.*, vol. 80, pp. 4973–4995, 2024. <https://doi.org/10.32604/cmc.2024.055839>
- [15] M. H. Wang, Y. Chen, and L. W. Kou, “Lightweight fish detection algorithm based on YOLOv8n,” *Mod. Electron. Tech.*, vol. 48, no. 5, pp. 79–85, 2025. <https://doi.org/10.16652/j.issn.1004-373x.2025.05.013>
- [16] S. Woo, J. Park, J. Y. Lee, and I. S. Kweon, “CBAM: Convolutional block attention module,” in *Computer Vision—ECCV 2018*. Springer, 2018, pp. 3–19. https://doi.org/10.1007/978-3-030-01234-2_1

- [17] L. Zhao, J. Liu, Y. Ren, C. Lin, J. Liu, Z. Abbas, S. Islam, and G. Xiao, "YOLOv8-QR: An improved YOLOv8 model via attention mechanism for object detection of QR code defects," *Comput. Electr. Eng.*, vol. 118, 2024. <https://doi.org/10.1016/j.compeleceng.2024.109376>
- [18] H. Chen, J. G. Yan, H. Yang, J. Zhang, W. Li, and J. Yang, "Deep separable convolutional neural networks based on structural reparameterization," *J. Beijing Univ. Aeronaut. Astronaut.*, pp. 1–15, 2024. <https://doi.org/10.13700/j.bh.1001-5965.2024.0287>
- [19] W. Liu, H. Lu, H. Fu, and Z. Cao, "Learning to upsample by learning to sample," in *2023 IEEE/CVF International Conference on Computer Vision (ICCV)*, Paris, France, 2023, pp. 6004–6014. <https://doi.org/10.1109/ICCV51070.2023.00554>
- [20] M. H. Guo, C. Z. Lu, Z. N. Liu, M. M. Cheng, and S. M. Hu, "Visual attention network," *Comput. Vis. Media*, vol. 9, pp. 733–752, 2023. <https://doi.org/10.1007/s41095-023-0364-2>
- [21] K. W. Lau, L. M. Po, and Y. A. U. Rehman, "Large Separable Kernel Attention: Rethinking the Large Kernel Attention design in CNN," *Expert Syst. Appl.*, vol. 236, 2023. <https://doi.org/10.1016/j.eswa.2023.121352>
- [22] S. L. Ma and Y. Xu, "MPDIoU: A loss for efficient and accurate bounding box regression," *arXiv preprint*, 2023, Art no. arXiv.2307.07662. <https://doi.org/10.48550/arXiv.2307.07662>
- [23] K. Han, Y. Wang, C. Xu, J. Guo, C. Xu, E. Wu, and Q. Tian, "Ghostnets on heterogeneous devices via cheap operations," *Int. J. Comput. Vis.*, vol. 130, pp. 1050–1069, 2022. <https://doi.org/10.1007/s11263-022-01575-y>